

PostgreSQL & Database Glossary — 150+ Terms

A comprehensive reference glossary covering PostgreSQL internals, SQL concepts, query optimization, replication, security, and production operations. Every term includes a definition, PostgreSQL context, example, and related terms.

Table of Contents

1. [How to Use This Glossary](#)
 2. [Core Database Concepts \(A-C\)](#)
 3. [Core Database Concepts \(D-G\)](#)
 4. [PostgreSQL Internals \(H-M\)](#)
 5. [Query Optimization \(N-Q\)](#)
 6. [Replication and HA \(R-S\)](#)
 7. [Production Operations \(T-Z\)](#)
 8. [Alphabetical Index](#)
-

1. How to Use This Glossary

Terms are grouped thematically, then alphabetically within each group. For a quick lookup, use the Alphabetical Index at the end.

Each entry follows this format:

- **Term** — Definition
 - *PostgreSQL context* — How this applies specifically in PostgreSQL
 - **Example** — A concrete SQL or command example
 - *Related terms* — Other glossary entries to read together
-

2. Core Database Concepts (A-C)

ACID

Definition: A set of four properties that guarantee database transactions are processed reliably: **A**tomicity, **C**onsistency, **I**solation, **D**urability.

Atomicity: A transaction is all-or-nothing. Either every statement in the transaction succeeds and is committed, or none of them are (the transaction is rolled back). There is no partial success.

Consistency: A transaction takes the database from one valid state to another valid state. All constraints (foreign keys, CHECK constraints, UNIQUE constraints) must be satisfied at commit time.

Isolation: Concurrent transactions behave as if they are executed serially. The degree of isolation is configurable (see Isolation Level).

Durability: Once a transaction is committed, it persists even if the server crashes immediately afterward. PostgreSQL implements this with WAL (Write-Ahead Logging).

PostgreSQL context: PostgreSQL is fully ACID-compliant. The default isolation level is Read Committed. `SERIALIZABLE` is available for full isolation.

```
BEGIN;
  UPDATE accounts SET balance = balance - 100 WHERE id = 1;
  UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
-- Atomicity: either both updates happen or neither does.
-- If the server crashes mid-transaction, WAL ensures rollback on restart.
```

Related terms: WAL, MVCC, Isolation Level, Transaction, Durability

Advisory Lock

Definition: A user-managed locking mechanism in PostgreSQL that does not lock any actual rows or tables. The application decides what the lock "means" — PostgreSQL just tracks whether it is held.

PostgreSQL context: Advisory locks are stored in shared memory and released at transaction or session end. Two variants: session-level (persist until explicitly released) and transaction-level (released at COMMIT/ROLLBACK).

```
-- Session-level: acquire lock with id 12345
SELECT pg_advisory_lock(12345);
-- ... do work ...
SELECT pg_advisory_unlock(12345);

-- Transaction-level
BEGIN;
  SELECT pg_advisory_xact_lock(12345);
  -- ... do work ...
COMMIT; -- lock released automatically

-- Non-blocking attempt
SELECT pg_try_advisory_lock(12345); -- returns TRUE or FALSE
```

Use case: Distributed cron jobs — only one pod should run a job at a time. Use an advisory lock to coordinate.

Related terms: Deadlock, Lock Contention, SELECT FOR UPDATE

Autovacuum

Definition: A background process in PostgreSQL that automatically runs VACUUM and ANALYZE on tables when they accumulate enough dead tuples or when statistics become stale.

PostgreSQL context: Autovacuum consists of a launcher and worker processes. The launcher wakes up periodically (controlled by `autovacuum_naptime`, default 60s) and spawns workers for tables that exceed thresholds.

Trigger threshold formula:

```
vacuum threshold = autovacuum_vacuum_threshold + autovacuum_vacuum_scale_factor *
n_live_tup
```

Default: $50 + 0.2 * n_live_tup$ — so vacuum triggers when dead tuples exceed 20% of live tuples plus 50.

```
-- Check autovacuum status for all tables
SELECT schemaname, tablename,
       last_autovacuum, last_autoanalyze,
       n_dead_tup, n_live_tup,
       round(n_dead_tup::numeric / nullif(n_live_tup,0) * 100, 2) AS dead_pct
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;

-- Override autovacuum settings per table
ALTER TABLE orders SET (
    autovacuum_vacuum_scale_factor = 0.01,    -- trigger at 1% dead tuples
    autovacuum_vacuum_cost_delay = 2          -- less throttling (ms)
);
```

Related terms: VACUUM, Bloat, Dead Tuple, Visibility Map, Free Space Map

Base Backup

Definition: A complete physical copy of the PostgreSQL data directory at a specific point in time. Used as the starting point for PITR (Point-In-Time Recovery) and as the initial data for a standby server.

PostgreSQL context: `pg_basebackup` creates a base backup. It can stream WAL during the backup to ensure consistency. The backup includes all data files, configuration files, and a backup label file.

```
# Create a base backup
pg_basebackup \
  -h localhost \
  -U replication_user \
  -D /var/lib/postgresql/16/backup \
  -Fp -Xs -P -R

# -Fp: plain format
# -Xs: stream WAL
# -P: show progress
# -R: write recovery configuration (for standby)
```

Related terms: PITR, WAL, Streaming Replication, pgBackRest

Bitmap Scan

Definition: A two-phase index scan strategy used when multiple index conditions need to be combined (AND/OR), or when a single index would return many rows. Phase 1 builds an in-memory bitmap of matching page addresses. Phase 2 reads only those heap pages.

PostgreSQL context: A Bitmap Index Scan reads the index and produces a bitmap. A Bitmap Heap Scan uses that bitmap to fetch rows. Multiple Bitmap Index Scans can be combined with `BitmapAnd` and `BitmapOr` operations before the heap fetch. This is more efficient than multiple separate Index Scans for high-selectivity queries.

```
EXPLAIN SELECT * FROM orders WHERE status = 'pending' AND customer_id = 42;
/*
Bitmap Heap Scan on orders
  Recheck Cond: ((status = 'pending') AND (customer_id = 42))
    -> BitmapAnd
          -> Bitmap Index Scan on orders_status_idx
          -> Bitmap Index Scan on orders_customer_id_idx
*/
```

Related terms: Index Scan, Sequential Scan, EXPLAIN, work_mem

Bloat

Definition: The accumulation of dead tuples, unused index entries, and wasted space in heap and index files that PostgreSQL cannot immediately reclaim.

PostgreSQL context: Bloat occurs because MVCC requires old tuple versions to remain on-disk until no active transaction can see them, and VACUUM only marks the space as reusable — it doesn't return it to the OS (unless the pages at the physical end of the file are freed). Index bloat occurs from deleted entries that leave gaps in B-Tree pages.

Causes:

- Long-running transactions preventing VACUUM from cleaning up
- Disabled or under-tuned autovacuum
- High UPDATE/DELETE rate without sufficient VACUUM frequency
- `VACUUM FULL` run while holding heavy locks prevents this becoming acute

```
-- Estimate table bloat
SELECT tablename,
       pg_size_pretty(pg_total_relation_size(relid)) AS total_size,
       pg_size_pretty(pg_relation_size(relid)) AS table_size,
       n_dead_tup,
       n_live_tup
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000
ORDER BY n_dead_tup DESC;

-- Reclaim bloat without full rewrite (online)
-- Install pg_repack extension
pg_repack -d mydb -t orders
```

Related terms: VACUUM, Autovacuum, Dead Tuple, MVCC, pg_repack

B-Tree Index

Definition: The default and most common index type in PostgreSQL, based on the Balanced Tree data structure. Efficient for equality lookups, range queries, ORDER BY, and `<` `>` `<=` `=` operators.

PostgreSQL context: B-Tree pages are 8KB by default. The tree maintains balance through page splits. PostgreSQL B-Trees support forward and backward scans. Height is typically 2–4 for most production tables.

Leaf pages contain sorted index entries pointing to heap tuple locations (ctid).

```
-- Default index type is B-Tree
CREATE INDEX ON orders (created_at);

-- Multi-column B-Tree (order matters: leftmost-prefix rule)
CREATE INDEX ON orders (customer_id, status, created_at);
-- This index supports: customer_id, (customer_id + status), (customer_id + status +
created_at)
-- NOT: status alone, created_at alone

-- Check B-Tree usage statistics
SELECT indexname, idx_scan, idx_tup_read, idx_tup_fetch
FROM pg_stat_user_indexes
WHERE tablename = 'orders';
```

Related terms: Index Scan, Bitmap Scan, GIN Index, GiST Index, BRIN Index

BRIN Index

Definition: Block Range INdex. A very small, approximate index that stores the minimum and maximum values for a range of heap pages (a block range). Only useful when column values are correlated with physical storage order.

PostgreSQL context: Each BRIN entry covers `pages_per_range` (default 128) consecutive heap pages. For time-series data inserted in chronological order, BRIN is extremely compact (kilobytes vs megabytes for B-Tree) and provides good filtering.

```
-- BRIN for a timestamp column inserted in order (e.g., event logs)
CREATE INDEX ON events USING BRIN (occurred_at);

-- Check if BRIN is being used
EXPLAIN SELECT * FROM events WHERE occurred_at > now() - interval '1 day';
```

When to use: Large tables, append-mostly, column physically ordered (e.g., `created_at` for time-series). **When NOT to use:** Randomly ordered data, random UUID primary keys, update-heavy workloads.

Related terms: B-Tree Index, GIN Index, Partial Index, Sequential Scan

Checkpoint

Definition: A process that flushes all dirty shared buffer pages to disk and writes a checkpoint record to WAL, establishing a known-good recovery point. After a checkpoint, WAL records before it are no longer needed for crash recovery.

PostgreSQL context: Checkpoints are performed by the checkpoint process. They happen on schedule (`checkpoint_timeout`, default 5 min) or when WAL grows beyond `max_wal_size` (default 1GB). The checkpoint process spreads I/O over `checkpoint_completion_target` (default 0.9) of the checkpoint interval to avoid I/O spikes.

```
-- Force a checkpoint manually (DBA use only)
CHECKPOINT;

-- Monitor checkpoint activity
SELECT checkpoints_timed, checkpoints_req, checkpoint_write_time,
       checkpoint_sync_time, buffers_checkpoint, buffers_clean
FROM pg_stat_bgwriter;

-- If checkpoints_req >> checkpoints_timed, increase max_wal_size
```

Related terms: WAL, BGWriter, shared_buffers, Crash Recovery, PITR

Clustering (CLUSTER command)

Definition: A PostgreSQL command that physically rewrites a table's heap pages to match the order of a specific index. After clustering, rows that are logically adjacent in the index are physically adjacent on disk.

PostgreSQL context: CLUSTER acquires an ACCESS EXCLUSIVE lock. It does not maintain order after subsequent writes — subsequent INSERTs/UPDATEs will scatter again. Must be re-run periodically. For ongoing physical ordering, consider BRIN indexes or partitioning.

```
-- Cluster orders table by customer_id for faster per-customer scans
CLUSTER orders USING orders_customer_id_idx;

-- Check if a table is clustered
SELECT relname, relhasindex
FROM pg_class WHERE relname = 'orders';
```

Related terms: B-Tree Index, BRIN Index, Sequential Scan, Bloat

Connection Pooling

Definition: The practice of maintaining a pool of pre-established database connections that can be reused by application clients, avoiding the overhead of creating new connections for every request.

PostgreSQL context: PostgreSQL forks a new OS process for every connection (not a thread). This makes connection creation expensive (~5–10ms) and each connection uses ~5–10MB of memory. For high-concurrency applications, direct connection counts in the hundreds can overwhelm the server. PgBouncer is the most widely used connection pool for PostgreSQL.

PgBouncer modes:

- **Session mode:** One server connection per client session. Safe for all features.
- **Transaction mode:** One server connection per transaction. More efficient. Incompatible with prepared statements, advisory locks (session), LISTEN/NOTIFY.
- **Statement mode:** One server connection per statement. Rarely used.

```
# pgbouncer.ini minimal config
[databases]
mydb = host=localhost port=5432 dbname=mydb
```

```
[pgbouncer]
listen_port = 6432
pool_mode = transaction
max_client_conn = 1000
default_pool_size = 20
```

Related terms: PgBouncer, Patroni, max_connections, Backend Process

Cost Estimation

Definition: The PostgreSQL query planner's mechanism for estimating how expensive (in arbitrary cost units) each possible query plan is, to choose the cheapest one.

PostgreSQL context: Costs are computed in units where `seq_page_cost = 1.0`. Common costs:

- `random_page_cost` = 4.0 (spinning disk) or 1.1 (SSD)
- `cpu_tuple_cost` = 0.01
- `cpu_index_tuple_cost` = 0.005
- `cpu_operator_cost` = 0.0025

The startup cost is the cost before the first row is returned. The total cost is the cost for all rows. The planner picks the plan with the lowest total cost.

```
-- See cost estimates
EXPLAIN SELECT * FROM orders WHERE customer_id = 42;
/*
Index Scan using orders_customer_id_idx on orders
 (cost=0.43..8.45 rows=12 width=156)
--      ^^^^ startup cost
--      ^^^^ total cost
--      ^^^^ estimated rows
*/

-- Tell planner about SSD storage
ALTER SYSTEM SET random_page_cost = 1.1;
SELECT pg_reload_conf();
```

Related terms: EXPLAIN, Statistics, Seq Scan, Index Scan, pg_stats

CTE (Common Table Expression)

Definition: A named temporary result set defined with the `WITH` clause that can be referenced in the main query. Improves readability and allows recursive queries.

PostgreSQL context: Before PostgreSQL 12, CTEs were always optimization fences (materialized). From PostgreSQL 12+, the planner can inline non-recursive CTEs. Use `WITH ... AS MATERIALIZED (...)` to force materialization, or `AS NOT MATERIALIZED` to force inlining.

```
-- Standard CTE
WITH monthly_revenue AS (
  SELECT date_trunc('month', created_at) AS month,
         sum(amount) AS revenue
```

```

FROM orders
WHERE status = 'completed'
GROUP BY 1
)
SELECT month,
       revenue,
       lag(revenue) OVER (ORDER BY month) AS prev_month_revenue,
       revenue - lag(revenue) OVER (ORDER BY month) AS change
FROM monthly_revenue
ORDER BY month;

-- Recursive CTE (org chart traversal)
WITH RECURSIVE org AS (
  -- Anchor: start with CEO
  SELECT id, name, manager_id, 0 AS depth
  FROM employees WHERE manager_id IS NULL
  UNION ALL
  -- Recursive: add direct reports
  SELECT e.id, e.name, e.manager_id, org.depth + 1
  FROM employees e
  JOIN org ON e.manager_id = org.id
)
SELECT * FROM org ORDER BY depth, name;

```

Related terms: Subquery, Window Function, Recursive Query, Lateral Join

3. Core Database Concepts (D–G)

Dead Tuple

Definition: A tuple (row version) in a PostgreSQL heap that is no longer visible to any active or future transaction, but has not yet been physically removed from the page.

PostgreSQL context: Dead tuples are created by DELETE and UPDATE operations. An UPDATE in PostgreSQL creates a new tuple version and marks the old one as dead (by setting xmax to the committing transaction ID). Dead tuples consume disk space and cause table and index bloat until VACUUM processes them.

```

-- Count dead tuples per table
SELECT relname, n_dead_tup, n_live_tup,
       round(100.0 * n_dead_tup / nullif(n_live_tup, 0), 2) AS dead_pct,
       last_autovacuum
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC
LIMIT 20;

```

Related terms: VACUUM, MVCC, Bloat, Autovacuum, xmin/xmax

Deadlock

Definition: A situation where two or more transactions are waiting for each other to release locks, creating a circular dependency that can never be resolved without external intervention.

PostgreSQL context: PostgreSQL detects deadlocks automatically using a deadlock detection algorithm (runs when `deadlock_timeout` elapses, default 1 second). When a deadlock is detected, PostgreSQL cancels one of the transactions (the victim) with an error and allows the other to proceed. The canceled transaction can be retried.

```
-- Session 1
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
-- (Session 2 now updates id=2, then tries to update id=1 – deadlock!)
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;

-- Prevention: always lock rows in the same order across all transactions
-- Use advisory locks as an alternative for complex coordination

-- Monitor active deadlocks in logs
-- postgresql.conf: log_lock_waits = on, deadlock_timeout = 1s
```

Related terms: Lock Contention, Advisory Lock, SELECT FOR UPDATE, pg_locks

Dirty Read

Definition: A transaction reads data written by a concurrent transaction that has not yet committed. If that transaction later rolls back, the reading transaction has seen data that never officially existed.

PostgreSQL context: PostgreSQL's MVCC implementation makes dirty reads impossible. Even at Read Uncommitted isolation level, PostgreSQL behaves like Read Committed — it never returns uncommitted data to any transaction. This is a deliberate PostgreSQL design decision.

```
-- PostgreSQL prevents this automatically – no action needed
-- The lowest isolation level you can set is Read Uncommitted, but it
-- actually behaves as Read Committed in PostgreSQL
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
-- Still won't see uncommitted data from other sessions
```

Related terms: Isolation Level, MVCC, Non-Repeatable Read, Phantom Read

Explain (EXPLAIN)

Definition: A PostgreSQL command that shows the query execution plan chosen by the planner without actually executing the query (unless ANALYZE is specified).

PostgreSQL context: `EXPLAIN` shows estimated costs. `EXPLAIN ANALYZE` executes the query and shows actual row counts and timing. `EXPLAIN (ANALYZE, BUFFERS)` additionally shows buffer hits/misses. `EXPLAIN (FORMAT JSON)` outputs machine-parseable JSON.

```
-- Basic plan
EXPLAIN SELECT * FROM orders WHERE customer_id = 42;
```

```

-- Full diagnostic
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT o.id, o.total, c.name
FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE o.created_at > now() - interval '7 days';

/*
Gather  (cost=1000.00..45231.12 rows=1523 width=72)
    (actual time=12.345..234.567 rows=1489 loops=1)
    Workers Planned: 2
    Workers Launched: 2
    Buffers: shared hit=8234 read=1204  -- 8234 from cache, 1204 from disk
    -> Hash Join  (cost=...)
        Hash Cond: (o.customer_id = c.id)
        -> Parallel Seq Scan on orders  ...
        -> Hash on customers  ...
Planning Time: 1.234 ms
Execution Time: 312.456 ms
*/

```

Related terms: Cost Estimation, Sequential Scan, Index Scan, pg_stat_statements, Statistics

FDW (Foreign Data Wrapper)

Definition: A PostgreSQL extension mechanism that allows querying external data sources (other PostgreSQL databases, MySQL, Oracle, files, REST APIs) as if they were local tables.

PostgreSQL context: The most common FDW is `postgres_fdw` for cross-PostgreSQL queries. `file_fdw` queries CSV files. `ogr_fdw` queries geospatial files. The SQL/MED standard defines the interface. PostgreSQL can push down WHERE clauses, column lists, and sometimes aggregate functions to the remote server.

```

-- Setup: connect two PostgreSQL servers
CREATE EXTENSION postgres_fdw;

CREATE SERVER remote_server
    FOREIGN DATA WRAPPER postgres_fdw
    OPTIONS (host 'analytics.internal', port '5432', dbname 'analytics');

CREATE USER MAPPING FOR current_user
    SERVER remote_server
    OPTIONS (user 'fdw_user', password 'secret');

-- Import the remote table structure
IMPORT FOREIGN SCHEMA public LIMIT TO (events)
    FROM SERVER remote_server INTO fdw_staging;

-- Query as local table
SELECT date_trunc('day', occurred_at) AS day, count(*)
FROM fdw_staging.events

```

```
WHERE occurred_at > now() - interval '30 days'
GROUP BY 1;
```

Related terms: Partitioning, Logical Replication, Data Engineering, postgres_fdw

Free Space Map (FSM)

Definition: A per-relation data structure that tracks the amount of free space available in each heap page, allowing PostgreSQL to efficiently find pages with enough room for new tuple insertions.

PostgreSQL context: The FSM is stored as a file with the `_fsm` suffix on the relation's file node. It is a tree structure that stores the maximum free space in byte buckets. VACUUM updates the FSM after cleaning dead tuples. Without a current FSM, PostgreSQL would scan pages sequentially to find free space, causing write amplification.

```
-- View FSM file path
SELECT pg_relation_filepath('orders');
-- e.g., base/16384/24601
-- FSM is: base/16384/24601_fsm

-- FSM info via pageinspect extension
SELECT * FROM pg_freespace('orders') LIMIT 10;
-- Returns (blkno, avail) pairs
```

Related terms: Visibility Map, VACUUM, Bloat, Heap File, Dead Tuple

GIN Index

Definition: Generalized Inverted Index. Efficient for columns that contain multiple values per row (arrays, JSONB, tsvector). The index maps each individual element to the list of rows containing it.

PostgreSQL context: GIN is the correct index type for: JSONB document queries (`@>` , `?` , `?|` , `?&`), array containment (`@>` , `&&`), and full-text search (`@@`). GIN indexes are larger and slower to build than B-Tree, but very fast for containment and search queries.

```
-- GIN for JSONB
CREATE INDEX ON events USING GIN (payload);

-- Query using GIN
SELECT * FROM events WHERE payload @> '{"type": "purchase"}';

-- GIN for full-text search
CREATE INDEX ON articles USING GIN (to_tsvector('english', content));

-- Search query
SELECT title FROM articles
WHERE to_tsvector('english', content) @@ to_tsquery('postgresql & performance');

-- GIN for arrays
```

```
CREATE INDEX ON products USING GIN (tags);
SELECT * FROM products WHERE tags @> ARRAY['postgres', 'database'];
```

Related terms: GiST Index, B-Tree Index, Full-Text Search, JSONB, tsvector

GiST Index

Definition: Generalized Search Tree. An extensible index framework that supports arbitrary data types and complex search predicates. Used for geometric types, range types, and network types.

PostgreSQL context: GiST is a framework, not a single algorithm. Each data type provides its own GiST implementation. Common uses: geometric queries (`&&` , `<<` , `@>`), range overlap queries (`&&` , `@>`), and nearest-neighbor searches with PostGIS.

```
-- GiST for range types (e.g., booking overlap detection)
CREATE TABLE bookings (
  id SERIAL,
  resource_id INT,
  during TSTZRANGE,
  EXCLUDE USING GIST (resource_id WITH =, during WITH &&)
);

-- The EXCLUDE constraint above uses a GiST index automatically
-- to prevent overlapping bookings for the same resource

-- GiST for geometric search
CREATE TABLE locations (
  id SERIAL,
  name TEXT,
  position POINT
);
CREATE INDEX ON locations USING GIST (position);
SELECT name FROM locations WHERE position <@ circle '((0,0), 10)';
```

Related terms: GIN Index, BRIN Index, Range Types, EXCLUDE Constraint

4. PostgreSQL Internals (H-M)

Heap File

Definition: The main data file for a PostgreSQL table. Rows (tuples) are stored in 8KB pages within this file. "Heap" in this context refers to the unordered collection of tuples, not the computer science data structure.

PostgreSQL context: Each table's heap file is named after the relation's file node OID (e.g., `base/16384/24601`). Large tables are split into 1GB segments (`24601` , `24601.1` , `24601.2` , etc.). TOAST data is stored in a separate heap file.

```
-- Find heap file path for a table
SELECT pg_relation_filepath('orders');
```

```
-- View page-level information (requires pageinspect)
CREATE EXTENSION pageinspect;
SELECT * FROM page_header(get_raw_page('orders', 0));
/*
lsn          | 0/1A2B3C4D -- WAL LSN of last change to this page
checksum     | 0           -- page checksum (if enabled)
flags        | 0
lower        | 120         -- start of free space
upper        | 7456        -- end of free space (free = upper - lower)
special      | 8192        -- start of special space
pagesize     | 8192
version      | 4
*/
```

Related terms: Page, Tuple, TOAST, Free Space Map, Visibility Map

Hot Standby

Definition: A PostgreSQL standby server that is open for read-only queries while simultaneously receiving and applying WAL from the primary. Named "hot" because it is accessible, unlike a "warm" standby that is not.

PostgreSQL context: Hot standby is enabled with `hot_standby = on` in `postgresql.conf` on the standby. Read queries on the standby may be canceled if they conflict with recovery (e.g., a VACUUM on the primary needs to remove a tuple version the standby query is still reading). `hot_standby_feedback = on` lets the standby tell the primary which transactions are still active, preventing this.

```
-- On the standby server
SHOW hot_standby; -- should be 'on'

-- Check if you're on a standby
SELECT pg_is_in_recovery(); -- returns TRUE on standby

-- Monitor lag on the primary
SELECT application_name,
       state,
       sent_lsn,
       write_lsn,
       flush_lsn,
       replay_lsn,
       (sent_lsn - replay_lsn) AS replication_lag_bytes
FROM pg_stat_replication;
```

Related terms: Streaming Replication, Replication Lag, Patroni, hot_standby_feedback

Isolation Level

Definition: A setting that controls how much one transaction is isolated from the effects of concurrent transactions. Higher isolation provides more correctness but less concurrency.

PostgreSQL context: PostgreSQL supports four isolation levels (SQL standard), but the two lower ones behave the same way because of MVCC:

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	Not possible (PG)	Possible	Possible
Read Committed (default)	Not possible	Possible	Possible
Repeatable Read	Not possible	Not possible	Not possible (PG)
Serializable	Not possible	Not possible	Not possible

```
-- Set isolation level for a transaction
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
-- or
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- Check current isolation level
SHOW transaction_isolation;

-- Use Serializable for financial operations requiring full isolation
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
  SELECT balance FROM accounts WHERE id = 1;
  -- Another transaction cannot change this balance and commit
  -- before this transaction commits (SSI prevents it)
  UPDATE accounts SET balance = balance + 100 WHERE id = 1;
COMMIT;
```

Related terms: MVCC, Dirty Read, Phantom Read, Serializable Snapshot Isolation

JSONB

Definition: A PostgreSQL-native binary JSON data type that stores JSON data in a decomposed binary format, enabling efficient operators and indexing. The "B" stands for "Binary."

PostgreSQL context: `JSONB` is almost always preferred over `JSON`. `JSON` stores the original text verbatim (preserves whitespace, duplicate keys, key order). `JSONB` decomposes JSON into a binary tree, removes duplicates, does not preserve key order, but enables GIN indexing and is much faster for operators.

```
-- Create a JSONB column
CREATE TABLE events (
  id BIGINT PRIMARY KEY,
  payload JSONB NOT NULL
);

-- Insert JSON
INSERT INTO events VALUES (1, '{"type": "click", "user_id": 42, "meta": {"x": 100, "y": 200}}');

-- Operators
SELECT payload -> 'type'           -- returns JSON: "click"
SELECT payload ->> 'type'          -- returns TEXT: click
```

```

SELECT payload #> '{meta,x}'      -- path operator: returns JSON 100
SELECT payload #>> '{meta,x}'    -- path operator: returns TEXT 100

-- Containment check (uses GIN index)
SELECT * FROM events WHERE payload @> '{"type": "click"}';

-- Key existence (uses GIN index)
SELECT * FROM events WHERE payload ? 'user_id';

-- Update a key
UPDATE events
SET payload = payload || '{"processed": true}':::jsonb
WHERE id = 1;

-- Remove a key
UPDATE events
SET payload = payload - 'meta'
WHERE id = 1;

-- GIN index for JSONB
CREATE INDEX ON events USING GIN (payload);

```

Related terms: GIN Index, JSON, hstore, Full-Text Search

Lock Contention

Definition: A situation where multiple transactions are competing for the same lock, causing some transactions to wait (block) rather than execute immediately. High lock contention degrades throughput and latency.

PostgreSQL context: Common sources of lock contention:

- DDL changes (`ALTER TABLE` , `CREATE INDEX`) require `ACCESS EXCLUSIVE` lock
- `VACUUM FULL` requires `ACCESS EXCLUSIVE` lock
- Long-running transactions holding row locks
- Batch updates on a small range of rows

```

-- Find blocking and blocked sessions
SELECT
  blocked.pid AS blocked_pid,
  blocked.query AS blocked_query,
  blocking.pid AS blocking_pid,
  blocking.query AS blocking_query,
  now() - blocked.query_start AS blocked_for
FROM pg_stat_activity blocked
JOIN pg_stat_activity blocking
  ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))
WHERE cardinality(pg_blocking_pids(blocked.pid)) > 0;

-- Kill a blocking session (caution!)
SELECT pg_terminate_backend(blocking_pid);

```

Related terms: Deadlock, Advisory Lock, SELECT FOR UPDATE, pg_locks, pg_blocking_pids

Logical Decoding

Definition: A mechanism in PostgreSQL that transforms the WAL stream (a binary physical format) into a logical, user-friendly stream of row-level changes (INSERT, UPDATE, DELETE with old and new values).

PostgreSQL context: Logical decoding requires `wal_level = logical`. Output plugins translate the WAL records. The built-in plugin is `pgoutput` (used by logical replication). `wal2json` is a popular plugin producing JSON output. Debezium uses logical decoding for CDC.

```
-- Create a replication slot with pgoutput decoder
SELECT pg_create_logical_replication_slot('my_slot', 'pgoutput');

-- Peek at changes (for wal2json plugin)
SELECT * FROM pg_logical_slot_peek_changes('my_slot', NULL, NULL,
    'include-xids', '1', 'pretty-print', '1');

-- Consume changes
SELECT * FROM pg_logical_slot_get_changes('my_slot', NULL, NULL);

-- Drop slot
SELECT pg_drop_replication_slot('my_slot');
```

Related terms: WAL, Replication Slot, Logical Replication, Debezium, CDC

Logical Replication

Definition: A replication method that replicates data at the row level (INSERT, UPDATE, DELETE on specific tables), allowing partial replication, cross-version replication, and selective filtering.

PostgreSQL context: Logical replication uses the publication/subscription model. A publisher creates a `PUBLICATION` (what to publish). A subscriber creates a `SUBSCRIPTION` (where to connect and what to subscribe to). PostgreSQL 16 added row filtering and column filtering to publications.

```
-- ON PRIMARY: create publication
CREATE PUBLICATION my_pub FOR TABLE orders, customers;
-- Or all tables:
CREATE PUBLICATION all_tables FOR ALL TABLES;

-- ON SUBSCRIBER: create subscription
CREATE SUBSCRIPTION my_sub
    CONNECTION 'host=primary.db dbname=mydb user=replicator password=secret'
    PUBLICATION my_pub;

-- Monitor subscription status
SELECT subname, received_lsn, latest_end_lsn
FROM pg_stat_subscription;

-- Check for conflicts
SELECT * FROM pg_stat_subscription_stats;
```


Related terms: Streaming Replication, Replication Slot, Logical Decoding, WAL

LSN (Log Sequence Number)

Definition: A 64-bit monotonically increasing integer that uniquely identifies a position in the PostgreSQL WAL stream. Used to track replication progress, recovery points, and WAL file positions.

PostgreSQL context: LSNs are displayed as two hex values separated by `/`, e.g., `0/1A2B3C4D`. The left side is the WAL segment number; the right is the offset within that segment.

```
-- Current WAL insert position
SELECT pg_current_wal_lsn();

-- Current WAL flush position (on primary)
SELECT pg_current_wal_flush_lsn();

-- How far behind is the standby?
SELECT
    pg_current_wal_lsn() - pg_last_wal_receive_lsn() AS receive_lag_bytes,
    pg_current_wal_lsn() - pg_last_wal_replay_lsn() AS replay_lag_bytes
-- (run on standby)

-- Convert LSN to WAL file name
SELECT pg_walfile_name('0/1A2B3C4D');
```

Related terms: WAL, Streaming Replication, Checkpoint, PITR

MVCC (Multi-Version Concurrency Control)

Definition: The concurrency control mechanism PostgreSQL uses to allow multiple transactions to read and write data simultaneously without blocking each other. Each transaction sees a consistent snapshot of the database as of a specific point in time.

PostgreSQL context: PostgreSQL implements MVCC by storing multiple versions of each row on the heap. Each tuple has `xmin` (transaction ID that created it) and `xmax` (transaction ID that deleted/updated it). A transaction's snapshot determines which row versions are visible to it.

Visibility rule: A tuple is visible to transaction T if:

- `xmin` is committed and is in T's snapshot (or = T's own xid)
- `xmax` is null, or uncommitted, or is in T's snapshot

```
-- See xmin and xmax for rows in a table
SELECT xmin, xmax, id, status FROM orders LIMIT 5;

-- After an UPDATE:
-- Old version: xmax = committing transaction's xid
-- New version: xmin = same transaction's xid

-- After a DELETE:
-- Tuple: xmax = committing transaction's xid, visible = false to future snapshots
```

```
-- Check current transaction ID
SELECT txid_current();
```

Related terms: Transaction, Isolation Level, Dead Tuple, VACUUM, xmin, xmax, Snapshot

5. Query Optimization (N-Q)

Non-Repeatable Read

Definition: A read anomaly where a transaction reads the same row twice and gets different results because another committed transaction modified the row between the two reads.

PostgreSQL context: Occurs at Read Committed isolation level. Prevented by Repeatable Read and Serializable levels.

```
-- Session 1 (Read Committed)
BEGIN;
SELECT price FROM products WHERE id = 1; -- returns 10.00

-- Session 2 commits an update
-- UPDATE products SET price = 15.00 WHERE id = 1; COMMIT;

SELECT price FROM products WHERE id = 1; -- returns 15.00 (non-repeatable!)
COMMIT;

-- Fix: use REPEATABLE READ
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
-- Now both reads return the same value
```

Related terms: Isolation Level, Dirty Read, Phantom Read, MVCC

Parallel Query

Definition: The ability for PostgreSQL to execute parts of a query using multiple worker processes simultaneously, reducing execution time for CPU-bound or large scan operations.

PostgreSQL context: Parallelism is enabled by `max_parallel_workers_per_gather` (default 2). Workers are spawned by Gather or Gather Merge nodes. Parallel scans divide the heap into chunks. Not all queries can be parallelized: those with row locks, functions marked `PARALLEL UNSAFE`, or certain aggregates.

```
-- Enable parallel query
SET max_parallel_workers_per_gather = 4;

-- Force parallel plan for testing
SET min_parallel_table_scan_size = 0;
SET parallel_setup_cost = 0;

-- Check parallel plan
EXPLAIN SELECT sum(amount) FROM large_orders;
/*
```

```

Finalize Aggregate (cost=...)
  -> Gather (cost=...)
      Workers Planned: 3
      -> Partial Aggregate (cost=...)
          -> Parallel Seq Scan on large_orders (cost=...)
*/

-- Mark a function as parallel-safe
CREATE FUNCTION safe_discount(price NUMERIC) RETURNS NUMERIC
  LANGUAGE sql
  PARALLEL SAFE
AS $$ SELECT price * 0.9; $$;

```

Related terms: Sequential Scan, EXPLAIN, work_mem, max_parallel_workers

Partial Index

Definition: An index built on a subset of a table's rows, defined by a WHERE clause. Only rows satisfying the WHERE predicate are indexed, making the index smaller and faster.

PostgreSQL context: Partial indexes are most useful for frequently queried subsets of data (e.g., unprocessed items, active users, recent orders). The query's WHERE clause must be compatible with the index's WHERE clause for the planner to use it.

```

-- Index only unprocessed orders (small fraction of the table)
CREATE INDEX ON orders (created_at) WHERE status = 'pending';

-- This query uses the partial index
EXPLAIN SELECT * FROM orders WHERE status = 'pending' AND created_at > now() -
interval '1 hour';

-- Partial unique index for soft deletes
CREATE UNIQUE INDEX ON users (email) WHERE deleted_at IS NULL;
-- Allows multiple deleted users with the same email, but only one active user

-- Partial index for non-null column
CREATE INDEX ON orders (customer_id) WHERE customer_id IS NOT NULL;

```

Related terms: B-Tree Index, Expression Index, Covering Index, GIN Index

Partitioning

Definition: Dividing a large table into smaller, physically separate sub-tables (partitions) that share the same schema but store disjoint subsets of the data.

PostgreSQL context: PostgreSQL supports declarative partitioning (PG 10+): RANGE, LIST, and HASH. Partition pruning means the planner skips irrelevant partitions in queries. Foreign keys cannot reference partitioned tables (as of PG 16, this is improving). Each partition can have its own indexes, tablespaces, and autovacuum settings.

```

-- Create a range-partitioned table by date
CREATE TABLE events (
    id BIGINT NOT NULL,
    occurred_at TIMESTAMPTZ NOT NULL,
    payload JSONB
) PARTITION BY RANGE (occurred_at);

-- Create monthly partitions
CREATE TABLE events_2024_01
    PARTITION OF events
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

CREATE TABLE events_2024_02
    PARTITION OF events
    FOR VALUES FROM ('2024-02-01') TO ('2024-03-01');

-- Verify partition pruning
EXPLAIN SELECT * FROM events WHERE occurred_at >= '2024-01-01' AND occurred_at <
'2024-02-01';
-- Should show only events_2024_01

-- Hash partitioning for even distribution
CREATE TABLE users (id BIGINT, email TEXT)
PARTITION BY HASH (id);

CREATE TABLE users_0 PARTITION OF users FOR VALUES WITH (MODULUS 4, REMAINDER 0);
CREATE TABLE users_1 PARTITION OF users FOR VALUES WITH (MODULUS 4, REMAINDER 1);
CREATE TABLE users_2 PARTITION OF users FOR VALUES WITH (MODULUS 4, REMAINDER 2);
CREATE TABLE users_3 PARTITION OF users FOR VALUES WITH (MODULUS 4, REMAINDER 3);

```

Related terms: FDW, BRIN Index, Data Engineering, Partition Pruning

Patroni

Definition: An open-source high-availability solution for PostgreSQL that uses a Distributed Configuration Store (DCS) such as etcd, consul, or ZooKeeper to manage leader election, automatic failover, and cluster configuration.

PostgreSQL context: Patroni is a Python-based daemon that runs alongside each PostgreSQL instance. The primary holds a leader lease in the DCS. If the primary fails to renew its lease, a standby is elected as the new primary. PgBouncer is typically placed in front of Patroni for connection routing.

```

# patroni.yml minimal example
scope: postgres_cluster
name: node1

restapi:
    listen: 0.0.0.0:8008
    connect_address: node1.example.com:8008

etcd3:

```

```
hosts: etcd1:2379,etcd2:2379,etcd3:2379

bootstrap:
  dcs:
    ttl: 30
    loop_wait: 10
    retry_timeout: 30
    maximum_lag_on_failover: 1048576 # 1MB

postgresql:
  listen: 0.0.0.0:5432
  connect_address: node1.example.com:5432
  data_dir: /var/lib/postgresql/16/data
  parameters:
    wal_level: replica
    hot_standby: 'on'
    max_wal_senders: 10
    max_replication_slots: 10
```

```
# patronictl commands
patronictl -c /etc/patroni.yml list           # show cluster status
patronictl -c /etc/patroni.yml switchover     # controlled primary swap
patronictl -c /etc/patroni.yml failover       # emergency failover
patronictl -c /etc/patroni.yml history        # timeline history
```

Related terms: Streaming Replication, etcd, PgBouncer, Replication Slot, Failover

PgBouncer

Definition: A lightweight PostgreSQL connection pooler that sits between application clients and the PostgreSQL server, multiplexing many client connections over a small number of server connections.

PostgreSQL context: PgBouncer is the industry standard connection pool for PostgreSQL. In transaction mode, a single server connection can serve hundreds of client connections, as long as they are not simultaneously in a transaction. Most high-traffic PostgreSQL deployments use PgBouncer.

```
# pgbouncer.ini
[databases]
myapp = host=127.0.0.1 port=5432 dbname=myapp

[pgbouncer]
listen_addr = 0.0.0.0
listen_port = 6432

# Pool mode
pool_mode = transaction           # most efficient for stateless apps

# Sizing
max_client_conn = 2000            # max simultaneous app connections
default_pool_size = 25            # server connections per user+database pair
min_pool_size = 5                 # maintain minimum connections
```

```

reserve_pool_size = 5           # emergency reserve

# Timeouts
server_idle_timeout = 600      # close idle server connections after 10min
client_idle_timeout = 0        # never close idle client connections
query_timeout = 0              # no per-query timeout (rely on pg)
pool_mode = transaction

```

Related terms: Connection Pooling, Patroni, max_connections, Session Mode, Transaction Mode

pg_hba.conf

Definition: PostgreSQL Host-Based Authentication configuration file. Controls which hosts can connect, which databases they can connect to, which users they can authenticate as, and what authentication method is used.

PostgreSQL context: PostgreSQL evaluates `pg_hba.conf` rules top-to-bottom, using the first matching rule. Changes require `pg_reload_conf()` or `SIGHUP` — no restart needed (unless `pg_ctl reload` is used).

```

# pg_hba.conf format:
# TYPE  DATABASE  USER  ADDRESS  METHOD

# Local socket connections: use peer auth (OS user = pg user)
local   all             postgres            peer

# Local socket connections for app user: password auth
local   myapp           myapp_user          scram-sha-256

# IPv4 connections from app servers: password auth
host    myapp           myapp_user  10.0.0.0/8  scram-sha-256

# Replication connections
host    replication     replicator  10.0.0.0/8  scram-sha-256

# Reject everything else
host    all             all         0.0.0.0/0   reject

```

```

-- Reload without restart
SELECT pg_reload_conf();

```

Related terms: pg_ident.conf, Authentication, SCRAM, Security, SSL

pg_ident.conf

Definition: PostgreSQL's ident map configuration file. Maps operating system usernames to PostgreSQL usernames, allowing peer and ident authentication methods to translate between OS identity and database identity.

PostgreSQL context: Used with `peer` authentication (Unix socket) and `ident` authentication (TCP). A row in `pg_ident.conf` maps `(map_name, os_username, pg_username)`. Referenced from `pg_hba.conf` with `map=<map_name>`.

```
# pg_ident.conf format:
# MAPNAME      SYSTEM-USERNAME    PG-USERNAME

# Map OS user 'alice' to PG user 'app_user'
appmap         alice              app_user

# Map OS user 'deploy' to PG user 'deploy_readonly'
deploymap      deploy            deploy_readonly
```

```
# pg_hba.conf using ident map
local  myapp  all  peer  map=appmap
```

Related terms: pg_hba.conf, Authentication, Peer Authentication

pg_stat_statements

Definition: A PostgreSQL extension that tracks statistics about all SQL statements executed, including total calls, total time, mean time, rows returned, and buffer usage. The most important performance monitoring tool in PostgreSQL.

PostgreSQL context: Must be loaded via `shared_preload_libraries = 'pg_stat_statements'` and `CREATE EXTENSION pg_stat_statements`. Queries are normalized (parameter values replaced with `$1`, `$2`, etc.) before hashing, so all executions of a parameterized query are aggregated.

```
-- Install
shared_preload_libraries = 'pg_stat_statements' -- in postgresql.conf (requires
restart)
CREATE EXTENSION pg_stat_statements;

-- Top 10 slowest queries by total time
SELECT
  round(total_exec_time::numeric, 2) AS total_ms,
  calls,
  round(mean_exec_time::numeric, 2) AS mean_ms,
  round(stddev_exec_time::numeric, 2) AS stddev_ms,
  rows,
  query
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;

-- Top queries by I/O
SELECT
  query,
  shared_blks_read + shared_blks_hit AS total_blocks,
  shared_blks_read AS disk_reads,
  round(100.0 * shared_blks_hit /
    nullif(shared_blks_read + shared_blks_hit, 0), 2) AS cache_hit_pct
FROM pg_stat_statements
ORDER BY disk_reads DESC
```

```
LIMIT 10;

-- Reset statistics
SELECT pg_stat_statements_reset();
```

Related terms: EXPLAIN, Query Optimization, auto_explain, Monitoring

Phantom Read

Definition: A read anomaly where a transaction executes the same range query twice and gets different rows the second time, because another committed transaction inserted or deleted rows matching the range between the two reads.

PostgreSQL context: Prevented by Repeatable Read (in PostgreSQL, not in the SQL standard) and Serializable isolation levels. At Read Committed, phantom reads are possible.

```
-- Session 1 (Read Committed)
BEGIN;
SELECT count(*) FROM orders WHERE amount > 1000; -- returns 5

-- Session 2 commits INSERT INTO orders (amount) VALUES (1500); COMMIT;

SELECT count(*) FROM orders WHERE amount > 1000; -- returns 6 (phantom!)
COMMIT;

-- Fix: use REPEATABLE READ or SERIALIZABLE
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
-- Both queries return the same count
```

Related terms: Isolation Level, Non-Repeatable Read, MVCC, Serializable

PITR (Point-In-Time Recovery)

Definition: The ability to restore a PostgreSQL database to any specific moment in the past, not just to the last backup. Achieved by replaying WAL records from a base backup up to the target time.

PostgreSQL context: PITR requires continuous WAL archiving. The `archive_command` copies completed WAL segments to an archive location. To recover, restore a base backup, configure `restore_command` to retrieve WAL segments from the archive, and set `recovery_target_time`.

```
# Step 1: Configure WAL archiving (postgresql.conf)
archive_mode = on
archive_command = 'cp %p /wal_archive/%f'

# Step 2: Take base backup
pg_basebackup -D /restore/base -Fp -Xs -P

# Step 3: Configure recovery (postgresql.conf / recovery.conf)
restore_command = 'cp /wal_archive/%f %p'
recovery_target_time = '2024-03-15 14:23:00'
recovery_target_action = 'promote'
```



```
# Step 4: Start PostgreSQL – it will replay WAL to the target time
pg_ctl start -D /restore/base
```

Related terms: WAL, Base Backup, Streaming Replication, pgBackRest, Checkpoint

6. Replication and HA (R-S)

Replication Lag

Definition: The delay between when a change is committed on the primary database and when that change is applied (replayed) on a standby replica.

PostgreSQL context: Replication lag has three components:

- **Write lag:** Time to receive WAL from primary to standby
- **Flush lag:** Time to flush received WAL to standby disk
- **Replay lag:** Time to apply (replay) flushed WAL on the standby

```
-- Measure replication lag (run on primary)
SELECT
  application_name,
  state,
  write_lag,
  flush_lag,
  replay_lag,
  pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS total_lag_size
FROM pg_stat_replication;

-- Measure lag in seconds (run on standby)
SELECT now() - pg_last_xact_replay_timestamp() AS replication_lag;
```

Alert threshold: Lag > 5 seconds typically requires investigation. Lag > 30 seconds may indicate the standby cannot keep up and needs `max_wal_size` increase or hardware upgrade.

Related terms: Streaming Replication, Replication Slot, Hot Standby, Patroni

Replication Slot

Definition: A persistent object in PostgreSQL that tracks the WAL consumption position for a specific replica or logical decoding consumer, ensuring that WAL is retained until that consumer has processed it.

PostgreSQL context: Replication slots prevent WAL deletion for a specific consumer. This is valuable for standbys that may be temporarily disconnected. **Danger:** An unused replication slot will cause unbounded WAL accumulation and can fill the disk. Monitor slot lag closely.

```
-- View replication slots
SELECT slot_name, slot_type, active, restart_lsn, confirmed_flush_lsn,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS lag
FROM pg_replication_slots;
```

```
-- Drop a slot that is no longer needed
SELECT pg_drop_replication_slot('old_standby_slot');

-- Alert: set max_slot_wal_keep_size to limit WAL retained by slots
-- postgresql.conf
max_slot_wal_keep_size = 10GB -- prevent runaway WAL accumulation
```

Related terms: Streaming Replication, Logical Decoding, Logical Replication, WAL

Row-Level Security (RLS)

Definition: A PostgreSQL feature that allows table-level policies to restrict which rows a given database user can see or modify. Policies are enforced transparently — the SQL code itself does not change.

PostgreSQL context: RLS must be explicitly enabled per table. Policies use `USING` (for SELECT/DELETE) and `WITH CHECK` (for INSERT/UPDATE). Superusers and table owners bypass RLS by default. `FORCE ROW LEVEL SECURITY` overrides this. `SECURITY DEFINER` functions may bypass RLS.

```
-- Enable RLS on a table
ALTER TABLE documents ENABLE ROW LEVEL SECURITY;

-- Policy: users can only see their own documents
CREATE POLICY user_isolation ON documents
  USING (owner_id = current_user_id());

-- Policy: multi-tenant isolation using a session variable
CREATE POLICY tenant_isolation ON events
  USING (tenant_id = current_setting('app.tenant_id')::INT);

-- Set the tenant context at session start
SET app.tenant_id = '42';
SELECT * FROM events; -- only returns events for tenant 42

-- Admin bypasses RLS
SET ROLE admin_user;
SELECT * FROM events; -- returns all events (admin has BYPASSRLS)
```

Related terms: Security, pg_hba.conf, SECURITY DEFINER, Privilege Escalation

Serializable Snapshot Isolation (SSI)

Definition: The algorithm PostgreSQL uses to implement the `SERIALIZABLE` isolation level. SSI detects serialization anomalies at runtime and aborts the transaction that would create the anomaly.

PostgreSQL context: SSI works by tracking read/write dependencies between transactions. If a cycle is detected (meaning there is no valid serial order for the transactions), one transaction is aborted with `ERROR: could not serialize access due to read/write dependencies`. The application must retry.

```
-- Use serializable for financial operations
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
  SELECT balance FROM accounts WHERE id = 42;
```

```

-- If another transaction concurrently changes this row and commits
-- before this transaction, this transaction will be aborted with:
-- ERROR: could not serialize access due to read/write dependencies
UPDATE accounts SET balance = balance + 100 WHERE id = 42;
COMMIT;

-- Application must handle serialization failures
LOOP
BEGIN
    BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
    -- ... do work ...
    COMMIT;
    EXIT; -- success, exit loop
EXCEPTION WHEN serialization_failure THEN
    -- retry
END;
END LOOP;

```

Related terms: Isolation Level, MVCC, Phantom Read, Transaction

Sequential Scan (Seq Scan)

Definition: A query execution plan node that reads every page of a table from beginning to end, checking each row against the query predicate.

PostgreSQL context: Sequential scans are not always bad. For large portions of a table (e.g., > 10–20% of rows), a sequential scan is often faster than an index scan because it reads pages sequentially (optimizing disk I/O) rather than randomly. The planner chooses between seq scan and index scan based on cost estimation.

```

-- Force a seq scan for testing
SET enable_indexscan = off;
SET enable_bitmapscan = off;
EXPLAIN SELECT * FROM orders WHERE amount > 100;
-- Shows: Seq Scan on orders

-- When to add an index: if selectivity is < 10% of rows
-- When NOT to: if selectivity is > 20%, seq scan may be faster

-- Check if a table is being fully scanned frequently
SELECT relname, seq_scan, idx_scan
FROM pg_stat_user_tables
WHERE seq_scan > idx_scan
AND n_live_tup > 10000 -- only large tables
ORDER BY seq_scan DESC;

```

Related terms: Index Scan, Bitmap Scan, Cost Estimation, EXPLAIN

Sharding

Definition: Horizontal partitioning of data across multiple independent database servers (shards), each holding a subset of the data, to scale writes and storage beyond what a single server can provide.

PostgreSQL context: PostgreSQL does not natively shard across multiple servers (like some NewSQL databases).
Options for PostgreSQL sharding:

- **Application-level sharding:** Application routes to shard based on shard key
- **Citus:** Extension that turns PostgreSQL into a distributed database
- **FDW-based:** Use foreign data wrappers to federate queries (limited)

```
-- Application-level sharding (logic in app code)
-- shard_id = user_id % num_shards
-- conn = get_connection(shard_id)

-- Citus: distribute a table
SELECT create_distributed_table('orders', 'customer_id');
-- All rows with the same customer_id go to the same shard

-- Bad sharding keys (hot shards):
-- - created_at (recent data all on one shard)
-- - status (most rows may be 'completed')
-- Good sharding keys:
-- - user_id (even distribution if UUIDs or high-cardinality IDs)
-- - customer_id (if customers are evenly active)
```

Related terms: Partitioning, FDW, Citus, Connection Pooling, Horizontal Scaling

Streaming Replication

Definition: PostgreSQL's physical replication mechanism where WAL records are continuously streamed from the primary to one or more standby servers, which replay them to stay in sync.

PostgreSQL context: Streaming replication operates at the block level — it transfers raw WAL records, not SQL. This means the standby must be the same major PostgreSQL version. The standby runs in "recovery" mode, continuously replaying WAL. `synchronous_commit` controls whether a commit waits for the standby to acknowledge receiving and flushing the WAL.

```
-- On primary: verify replication is active
SELECT * FROM pg_stat_replication;

-- postgresql.conf (primary)
wal_level = replica
max_wal_senders = 10
max_replication_slots = 10

-- postgresql.conf (standby)
hot_standby = on
primary_conninfo = 'host=primary.db user=replicator password=secret'
primary_slot_name = 'standby1_slot'

-- Synchronous replication (wait for at least 1 standby to flush)
synchronous_standby_names = 'ANY 1 (standby1, standby2)'
synchronous_commit = on
```

Related terms: WAL, Replication Slot, Hot Standby, Patroni, Logical Replication, Replication Lag

System Catalog

Definition: A set of PostgreSQL internal tables (in the `pg_catalog` schema) that store all metadata about the database: tables, columns, indexes, constraints, types, functions, roles, and more.

PostgreSQL context: The system catalog is the authoritative source of truth about database structure.

`information_schema` is a SQL-standard view layer on top of `pg_catalog`. Direct `pg_catalog` access is faster and more detailed.

```
-- List all tables in current database
SELECT relname, relkind, relpages, reltuples
FROM pg_class
WHERE relkind = 'r' -- r = ordinary table
AND relnamespace = 'public'::regnamespace
ORDER BY relpages DESC;

-- List all indexes with their sizes
SELECT i.relname AS index_name, t.relname AS table_name,
       pg_size_pretty(pg_relation_size(i.oid)) AS index_size
FROM pg_index ix
JOIN pg_class t ON t.oid = ix.indrelid
JOIN pg_class i ON i.oid = ix.indexrelid
ORDER BY pg_relation_size(i.oid) DESC;

-- Column metadata
SELECT attname, atttypid::regtype, attnotnull, atthasdef
FROM pg_attribute
WHERE attrelid = 'orders'::regclass
AND attnum > 0; -- skip system columns
```

Related terms: `information_schema`, `pg_stat_user_tables`, `pg_stat_statements`, OID

7. Production Operations (T-Z)

TOAST (The Oversized-Attribute Storage Technique)

Definition: PostgreSQL's mechanism for storing large column values (text, bytea, JSONB, arrays) that exceed the 2KB inline threshold. Large values are compressed and/or stored in a separate TOAST table.

PostgreSQL context: Every table with potentially large columns has a corresponding TOAST table (named `pg_toast.pg_toast_<oid>`). The threshold is 2KB (roughly). TOAST strategies per column:

- `PLAIN` : No compression or out-of-line storage (for small types)
- `EXTENDED` : Try compression, then out-of-line (default for TEXT, JSONB)
- `EXTERNAL` : Out-of-line without compression (fast for random-access)
- `MAIN` : Try compression first, but prefer inline

```

-- Check TOAST strategy for each column
SELECT attname, attstorage, atttypid::regtype
FROM pg_attribute
WHERE attrelid = 'articles'::regclass
AND attnum > 0;
-- attstorage: 'p'=PLAIN, 'e'=EXTERNAL, 'x'=EXTENDED, 'm'=MAIN

-- Change strategy for faster reads (at cost of more storage)
ALTER TABLE articles ALTER COLUMN content SET STORAGE EXTERNAL;

-- TOAST table for a relation
SELECT relname FROM pg_class
WHERE relname = 'pg_toast_' || 'articles'::regclass::oid::text;

```

Related terms: Heap File, Tuple, VACUUM, Bloat

Transaction

Definition: A logical unit of work that groups one or more SQL statements so they either all succeed (COMMIT) or all fail (ROLLBACK) together. Transactions provide the ACID guarantees.

PostgreSQL context: Every SQL statement in PostgreSQL runs in a transaction, even if not explicitly wrapped in `BEGIN`. A single statement is an implicit single-statement transaction. Long-running transactions are dangerous because they hold locks, prevent VACUUM, and increase the risk of serialization conflicts.

```

-- Explicit transaction
BEGIN;
  INSERT INTO orders (customer_id, amount) VALUES (42, 99.99);
  UPDATE inventory SET quantity = quantity - 1 WHERE product_id = 7;
  -- If any statement fails, ROLLBACK is called
COMMIT;

-- Savepoints for partial rollback
BEGIN;
  INSERT INTO audit_log (action) VALUES ('batch_start');
  SAVEPOINT before_risky_operation;
  DELETE FROM temp_data WHERE batch_id = 123;
  -- If this fails, rollback to savepoint
  ROLLBACK TO SAVEPOINT before_risky_operation;
  INSERT INTO audit_log (action) VALUES ('batch_failed');
COMMIT;

-- Monitor long-running transactions
SELECT pid, now() - xact_start AS duration, query
FROM pg_stat_activity
WHERE state != 'idle'
AND xact_start < now() - interval '5 minutes'
ORDER BY duration DESC;

```

Related terms: ACID, MVCC, Isolation Level, Savepoint, Deadlock

VACUUM

Definition: A PostgreSQL maintenance operation that reclaims storage space occupied by dead tuples, updates statistics, and sets the visibility map, enabling Index Only Scans and preventing transaction ID wraparound.

PostgreSQL context: There are several forms:

- `VACUUM table` : Marks dead tuple space as reusable. Does NOT shrink the file.
- `VACUUM ANALYZE table` : `VACUUM` + update statistics.
- `VACUUM FULL table` : Complete table rewrite. Reclaims space to OS. Requires `ACCESS EXCLUSIVE` lock.
- `VACUUM FREEZE table` : Force-freezes old transaction IDs to prevent wraparound.
- `VACUUM VERBOSE table` : Prints detailed progress information.

```
-- Regular VACUUM (safe for production, low impact)
VACUUM orders;

-- With analysis
VACUUM ANALYZE orders;

-- Full vacuum (dangerous! takes exclusive lock, use pg_repack instead)
VACUUM FULL orders;

-- Freeze to prevent XID wraparound
VACUUM FREEZE orders;

-- Verbose output for debugging
VACUUM VERBOSE orders;
/*
INFO:  vacuuming "public.orders"
INFO:  scanned index "orders_pkey" to remove 1234 row versions
INFO:  "orders": removed 1234 row versions in 89 pages
INFO:  "orders": found 1234 removable, 9876 nonremovable row versions
...
*/

-- Monitor VACUUM progress (PG 13+)
SELECT * FROM pg_stat_progress_vacuum;
```

Related terms: Autovacuum, Bloat, Dead Tuple, Visibility Map, Free Space Map, Transaction ID Wraparound

Visibility Map (VM)

Definition: A per-table data structure with 2 bits per heap page indicating: (1) whether all tuples on the page are visible to all active transactions (all-visible bit), and (2) whether all tuples are frozen (all-frozen bit).

PostgreSQL context: The visibility map enables Index Only Scans: if the all-visible bit is set for a page, PostgreSQL knows it doesn't need to visit the heap to check visibility — the index alone provides sufficient information. `VACUUM` sets the all-visible bit. Any `UPDATE` or `DELETE` on a page clears the bit.

```

-- Check visibility map status
SELECT relname,
       n_live_tup,
       all_visible, -- pages where all tuples are visible
       all_frozen   -- pages where all tuples are frozen
FROM pg_class
JOIN pg_visibility_summary(oid) ON true
WHERE relname = 'orders';

-- Index Only Scan is possible only when visibility map is populated
EXPLAIN SELECT id FROM orders WHERE customer_id = 42;
/*
Index Only Scan using orders_customer_id_idx on orders
  (cost=0.43..4.45 rows=12 width=8)
  Heap Fetches: 0 -- no heap access needed!
*/

```

Related terms: VACUUM, Free Space Map, Index Only Scan, Bloat

WAL (Write-Ahead Log)

Definition: The primary mechanism PostgreSQL uses to ensure durability and enable crash recovery. Every change to data is first written to WAL (a sequential log on disk) before the corresponding heap/index pages are modified.

PostgreSQL context: WAL provides "write-ahead" guarantee: if the server crashes, PostgreSQL can replay WAL records from the last checkpoint to reconstruct any committed changes. WAL is also the basis for streaming replication (physical) and logical decoding (logical). WAL files are stored in `$PGDATA/pg_wal/` and are 16MB each by default.

```

-- Current WAL write position
SELECT pg_current_wal_lsn();

-- Current WAL file
SELECT pg_walfile_name(pg_current_wal_lsn());

-- WAL configuration (postgresql.conf)
-- wal_level = replica (or logical for CDC)
-- wal_compression = on (compress WAL records, reduces I/O)
-- wal_buffers = 16MB (WAL buffer in shared memory)
-- max_wal_size = 1GB (trigger checkpoint above this size)
-- min_wal_size = 80MB (keep at least this much WAL space)
-- archive_mode = on (enable WAL archiving)
-- archive_command = 'cp %p /wal_archive/%f'

-- WAL statistics
SELECT * FROM pg_stat_wal;

```

Related terms: Checkpoint, Streaming Replication, PITR, LSN, Logical Decoding, Crash Recovery

Window Function

Definition: A SQL function that performs calculations across a set of table rows related to the current row (the "window"), without collapsing rows into groups as GROUP BY does.

PostgreSQL context: Window functions use the `OVER()` clause. PostgreSQL implements: ranking functions (`ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`), offset functions (`LEAD`, `LAG`, `FIRST_VALUE`, `LAST_VALUE`, `NTH_VALUE`), and aggregate functions (`SUM`, `AVG`, `COUNT`, `MIN`, `MAX` as window functions).

```
-- Running total of order amounts per customer
SELECT
  customer_id,
  order_date,
  amount,
  SUM(amount) OVER (
    PARTITION BY customer_id
    ORDER BY order_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS running_total
FROM orders;

-- 7-day moving average
SELECT
  order_date,
  daily_revenue,
  AVG(daily_revenue) OVER (
    ORDER BY order_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
  ) AS seven_day_avg
FROM daily_revenue_summary;

-- Rank customers by revenue, partition by region
SELECT
  customer_id,
  region,
  total_revenue,
  RANK() OVER (PARTITION BY region ORDER BY total_revenue DESC) AS regional_rank
FROM customer_summary;
```

Related terms: CTE, Aggregate Function, GROUP BY, Lateral Join

xmin / xmax

Definition: System columns present in every PostgreSQL table row (tuple). `xmin` is the transaction ID of the transaction that inserted the row. `xmax` is the transaction ID of the transaction that deleted or updated the row (0 if the row is live and not deleted).

PostgreSQL context: These are the core of PostgreSQL's MVCC implementation. When evaluating which row version to show to a transaction, PostgreSQL checks whether `xmin` is committed and in the transaction's snapshot, and whether `xmax` is committed. A tuple is visible if `xmin` is committed-and-visible AND (`xmax` is 0 or uncommitted or post-snapshot).

```
-- See system columns
SELECT xmin, xmax, ctid, id, status FROM orders LIMIT 5;

-- After update: old tuple gets xmax set, new tuple gets new xmin
BEGIN;
UPDATE orders SET status = 'shipped' WHERE id = 1;
-- In a separate session, you'd see:
-- Old tuple: xmin=original_xid, xmax=current_xid (not yet committed)
-- New tuple: xmin=current_xid, xmax=0
COMMIT;

-- After commit: old tuple is dead (xmax = committed xid)
-- VACUUM will eventually remove the old tuple
```

Related terms: MVCC, Dead Tuple, VACUUM, Transaction, Snapshot

Zero-Downtime Migration

Definition: A database schema change that can be applied to a production database while the application continues to serve traffic without errors or outages.

PostgreSQL context: Many DDL operations in PostgreSQL require an `ACCESS EXCLUSIVE` lock, blocking all reads and writes. True zero-downtime migrations require careful sequencing.

```
-- SAFE: Add a nullable column (instant, no lock contention)
ALTER TABLE orders ADD COLUMN processed_at TIMESTAMPTZ;

-- SAFE: Add an index concurrently (no table lock, runs in background)
CREATE INDEX CONCURRENTLY idx_orders_processed_at ON orders (processed_at);

-- UNSAFE: Add NOT NULL column without default (rewrites table!)
-- ALTER TABLE orders ADD COLUMN is_processed BOOLEAN NOT NULL DEFAULT FALSE;
-- FIX: Three-step process for large tables:
-- Step 1: Add nullable column
ALTER TABLE orders ADD COLUMN is_processed BOOLEAN;
-- Step 2: Backfill (in batches!)
UPDATE orders SET is_processed = FALSE WHERE is_processed IS NULL;
-- Step 3: Add NOT NULL constraint (fast if no NULLs exist, PG 12+)
ALTER TABLE orders ALTER COLUMN is_processed SET NOT NULL;

-- UNSAFE: ALTER TABLE without CONCURRENTLY (holds AccessShareLock at minimum)
-- For dropping a column: it's "instant" (just marks as dropped), but verify
-- For renaming a column: consider the application deployment order

-- Best tool for complex zero-downtime migrations: pg_repack
```

Related terms: DDL, Lock Contention, Bloat, Autovacuum, Partitioning

8. Alphabetical Index

Term	Section
ACID	§2
Advisory Lock	§2
Autovacuum	§2
Base Backup	§2
Bitmap Scan	§2
Bloat	§2
BRIN Index	§2
B-Tree Index	§2
Checkpoint	§2
Clustering (CLUSTER)	§2
Connection Pooling	§2
Cost Estimation	§2
CTE	§2
Dead Tuple	§3
Deadlock	§3
Dirty Read	§3
EXPLAIN	§3
FDW	§3
Free Space Map	§3
GIN Index	§3
GiST Index	§3
Heap File	§4
Hot Standby	§4
Isolation Level	§4
JSONB	§4
Lock Contention	§4
Logical Decoding	§4
Logical Replication	§4
LSN	§4

MVCC	§4
Non-Repeatable Read	§5
Parallel Query	§5
Partial Index	§5
Partitioning	§5
Patroni	§5
PgBouncer	§5
pg_hba.conf	§5
pg_ident.conf	§5
pg_stat_statements	§5
Phantom Read	§5
PITR	§5
Replication Lag	§6
Replication Slot	§6
Row-Level Security	§6
Serializable Snapshot Isolation	§6
Sequential Scan	§6
Sharding	§6
Streaming Replication	§6
System Catalog	§6
TOAST	§7
Transaction	§7
VACUUM	§7
Visibility Map	§7
WAL	§7
Window Function	§7
xmin / xmax	§7
Zero-Downtime Migration	§7

See [10_PostgreSQL_Internals](#) for deep dives into WAL, MVCC, VACUUM, and heap file structure. See [08_Query_Optimization](#) for detailed EXPLAIN analysis and planner internals. See [13_Replication_HA](#) for

streaming replication and Patroni deployment. See `09_Transactions_Concurrency` for isolation levels and locking.